Faculty of Computing

Semester II, Session

2025/26

SECJ3483 SEC01

WEB TECHNOLOGY

GROUP LAB ASSIGNMENT

SECURITY INVESTIGATION AND FIX

FRONTEND, BACKEND SECURITY, API SECURITY AND JWT AUTHENTICATION

CASE STUDY: SECURE PERSON BMI FULL-STACK WEB APPLICATION

Name	**Matric No**
Chang Wei Lam	A23CS0212
Yap Jia Xin	A23CS0199
Karen Voon Xiu Wen	A23CS0229
Goh Chang Zhe	A23CS0225

1. Security Investigation Summary

The Person BMI application was intentionally designed with multiple security vulnerabilities to provide students with hands-on experience in identifying and mitigating common web application security issues. The investigation focused on input validation, authentication, authorization, SQL Injection, Cross-Site Scripting (XSS), sensitive data exposure, and insecure API design.

API endpoints were tested using Thunder Client and the frontend application. The investigation confirmed that the application accepted invalid input, trusted client-controlled fields, exposed sensitive user information, lacked proper role-based and owner-based access control, and was vulnerable to SQL Injection and Cross-Site Scripting. Appropriate security controls were then implemented and the application was re-tested to verify that the vulnerabilities had been resolved.

2. Weaknesses Found

2.1 Investigation Table

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data	Backend should reject invalid data	Backend accepted empty name, negative age, zero height, negative weight, client-supplied BMI and user_id, and created the record (201 Created).	Yes	No server-side validation. The backend trusted all client input, allowing invalid data and protected fields to be stored.
2	Access another user's BMI record	Backend should deny access	A normal user was able to request another user's BMI record by changing the record ID (200 OK).	Yes	Missing owner-based access control (IDOR/Broken Object Level Authorization). The API did not verify ownership before returning the record.
3	Access staff route as normal user	Backend should deny access	A normal user successfully accessed /api/staff/persons (200 OK).	Yes	Missing Role-Based Access Control (RBAC). The backend did not verify the user's role before allowing access.

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
4	Access admin route as normal user	Backend should deny access	A normal user successfully accessed /api/admin/users and retrieved all users (200 OK).	Yes	Missing RBAC on administrator endpoints. Sensitive user information was exposed to unauthorized users.
5	Submit XSS payload in notes	Payload should display as text	HTML payload (<img src=x onerror=alert(1)>) was stored and rendered using v-html, allowing browser execution of user-controlled HTML.	Yes	The frontend used v-html, which renders untrusted HTML directly and enables Cross-Site Scripting (XSS).
6	Check API response	Password/hash should not be visible	Login response exposed password, password_hash, and internal debug information.	Yes	Sensitive information disclosure. Authentication data should never be returned to clients.
7	Try SQL Injection input	Input should be treated as data	Login succeeded using aiman@student.utm.my' -- with an incorrect password. The SQL query was modified and the password check was bypassed.	Yes	SQL Injection vulnerability caused by string concatenation instead of prepared statements.
8	Try to update protected fields	Backend should reject or ignore protected fields	Client successfully changed user_id, bmi, and category through the update request (200 OK).	Yes	Mass Assignment / Unsafe Field Update. The backend trusted protected fields supplied by the client.

2.2 Classify the Weaknesses

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Negative weight accepted	Backend	Missing backend validation	Invalid data stored
User accesses another BMI record	API / Backend	Broken Object Level Authorization	Privacy violation
Normal user accesses admin route	API / Backend	Broken Function Level Authorization	Unauthorized admin action
API returns password_hash	API / Backend	Sensitive data exposure	Credential risk
XSS payload executes	Frontend	XSS	Browser script execution
SQL Injection input affects query	Backend / Database	SQL Injection	Unauthorized data access
User changes user_id or role	API / Backend	Mass assignment	Privilege or ownership manipulation

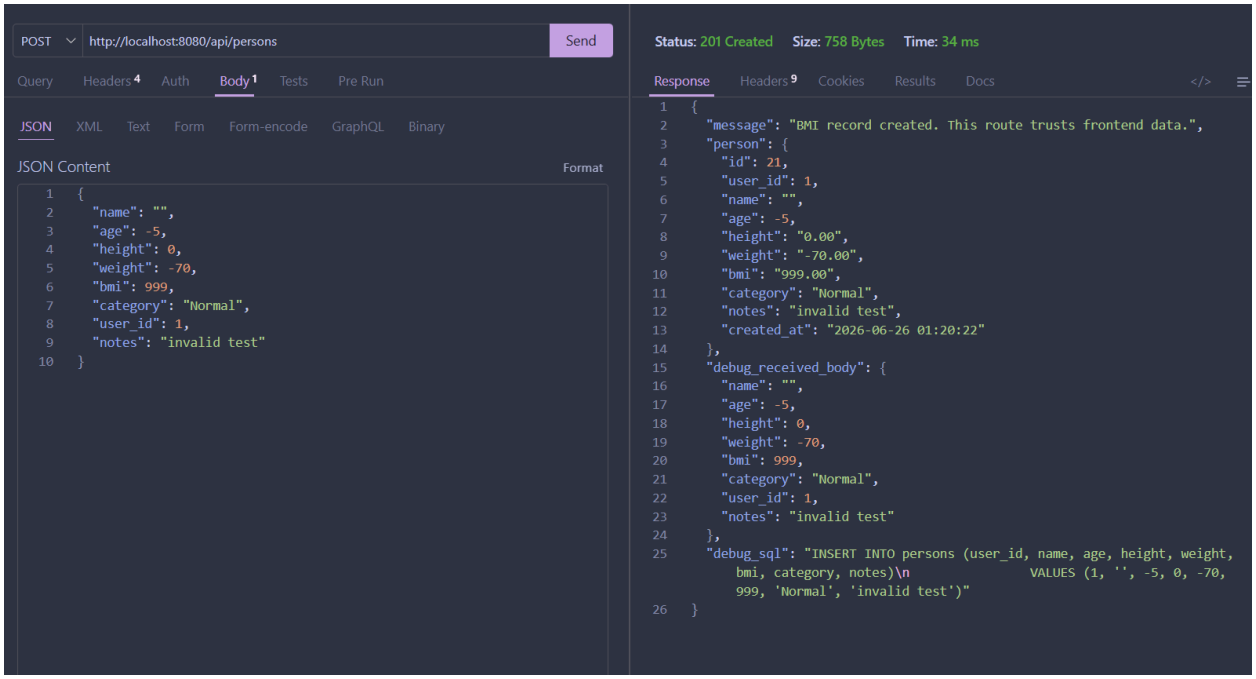
3. Fixes Implemented

Weakness	Fix Implemented
Missing validation	Added backend validation for all BMI input fields
SQL Injection	Replaced string concatenation with prepared statements
Plain password storage	Used `password_hash()` and `password_verify()`
Fake token	Implemented signed JWT authentication
Missing authorization	Added JWT authentication middleware
IDOR	Verified record ownership before returning or updating records
Missing RBAC	Restricted staff and admin routes based on user role
Sensitive data exposure	Removed password and password_hash from API responses
Mass Assignment	Ignored client-supplied user_id, bmi and category
XSS	Replaced `v-html` with Vue interpolation (`{{ person.notes }}`)
Error handling	Returned generic error messages without exposing SQL details

4. Testing evidence

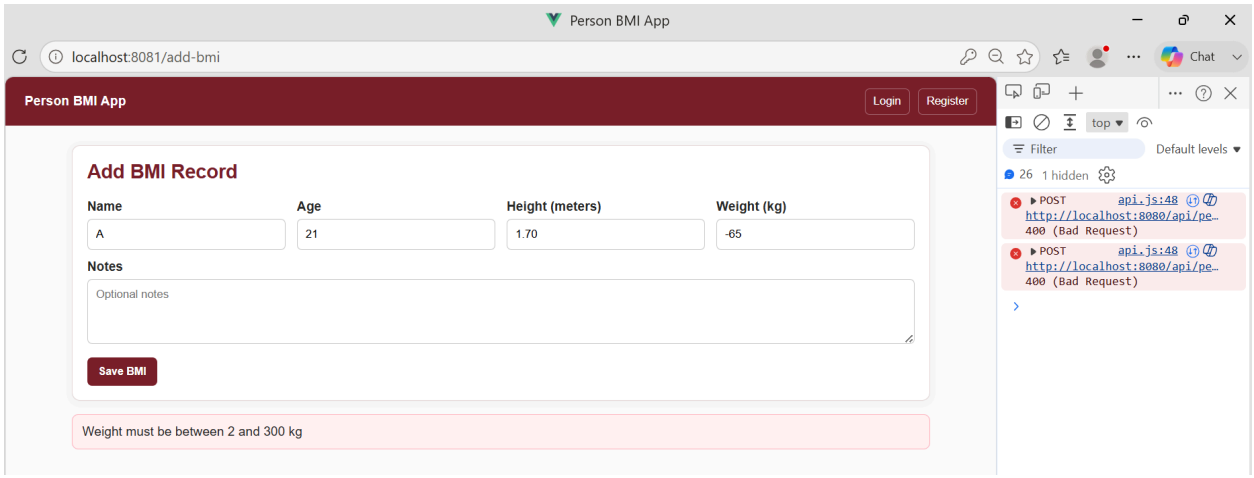
Test 1: Add BMI with invalid data

BEFORE:



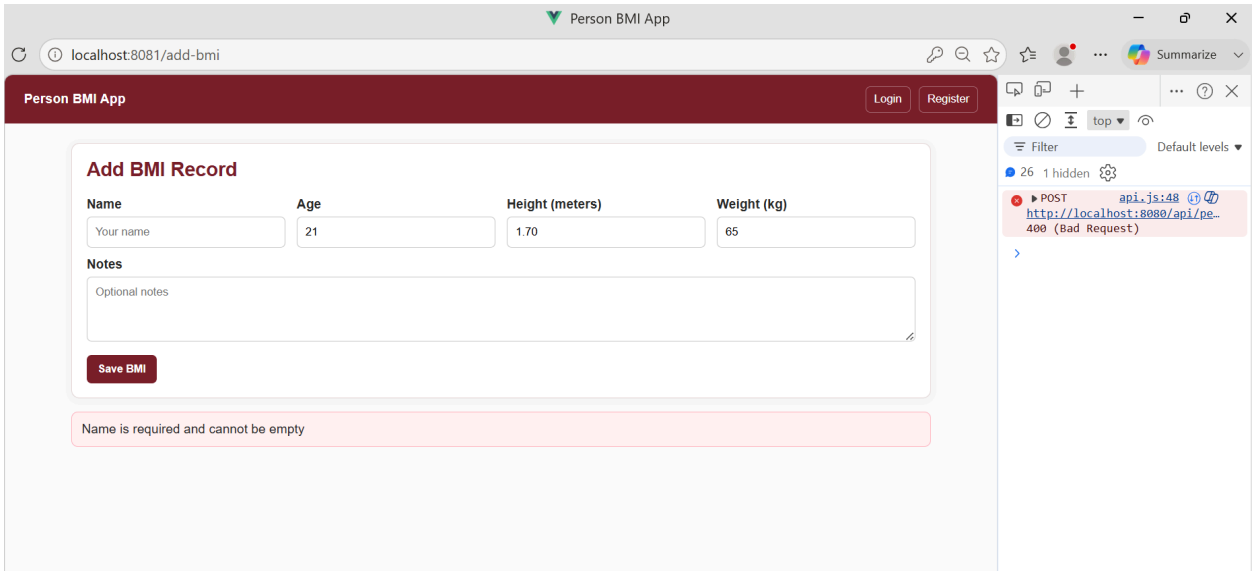
AFTER FIX:

1. BMI with negative weight
   After Fix - 400 Bad Request



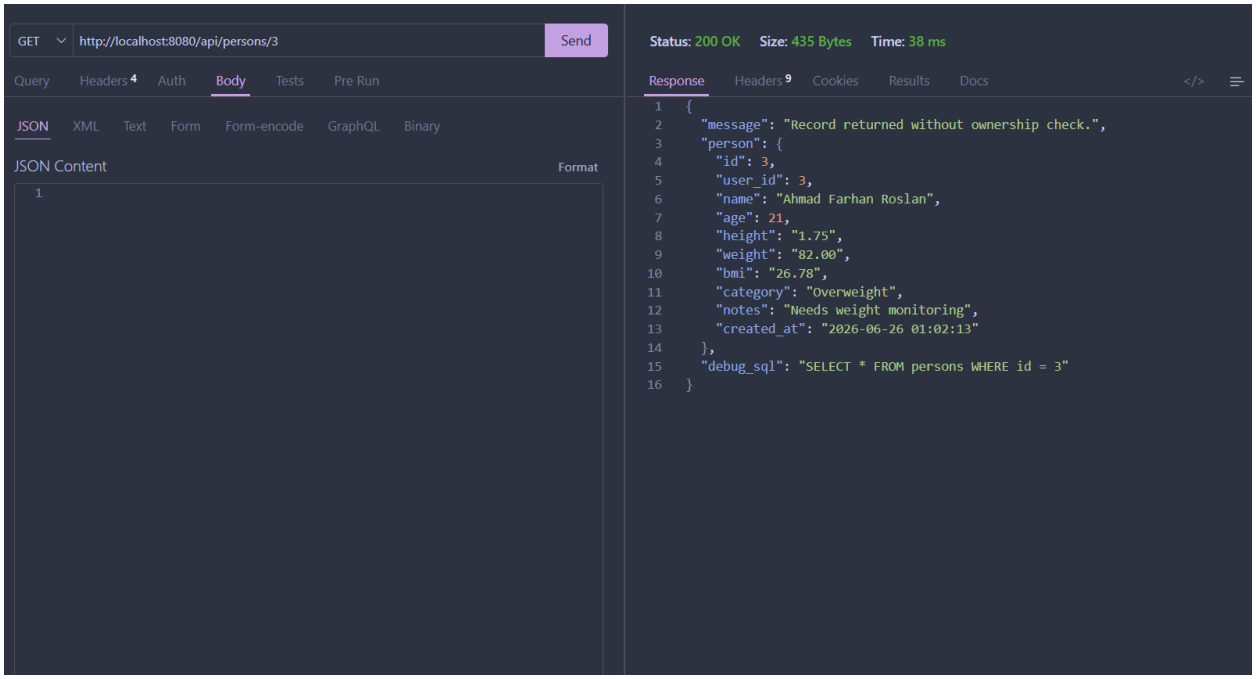
2. Submit empty name

After Fix- 400 Bad Request



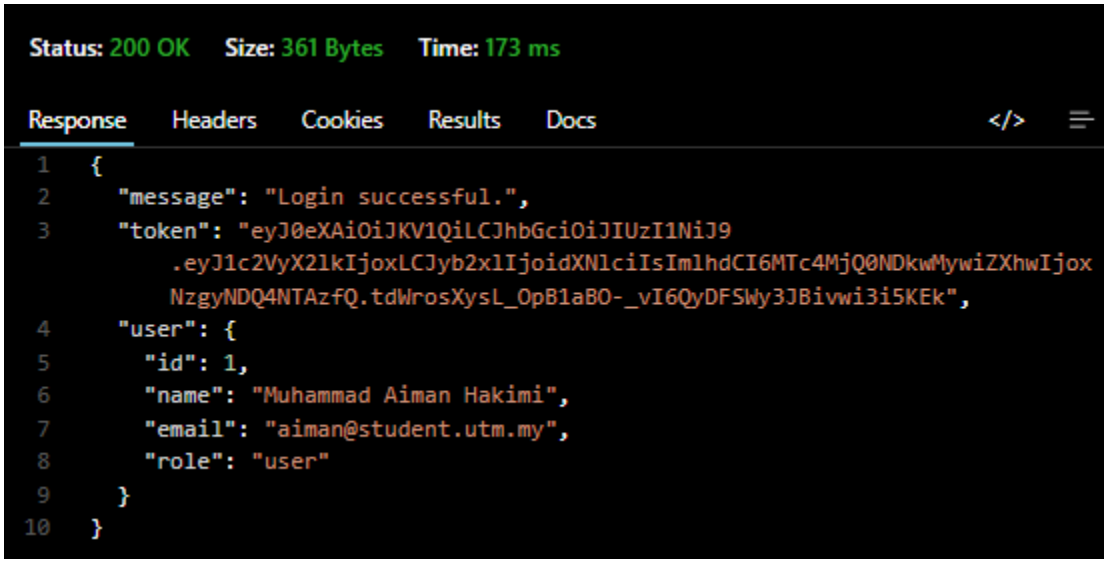
Test 2: Access another user BMI record

BEFORE:

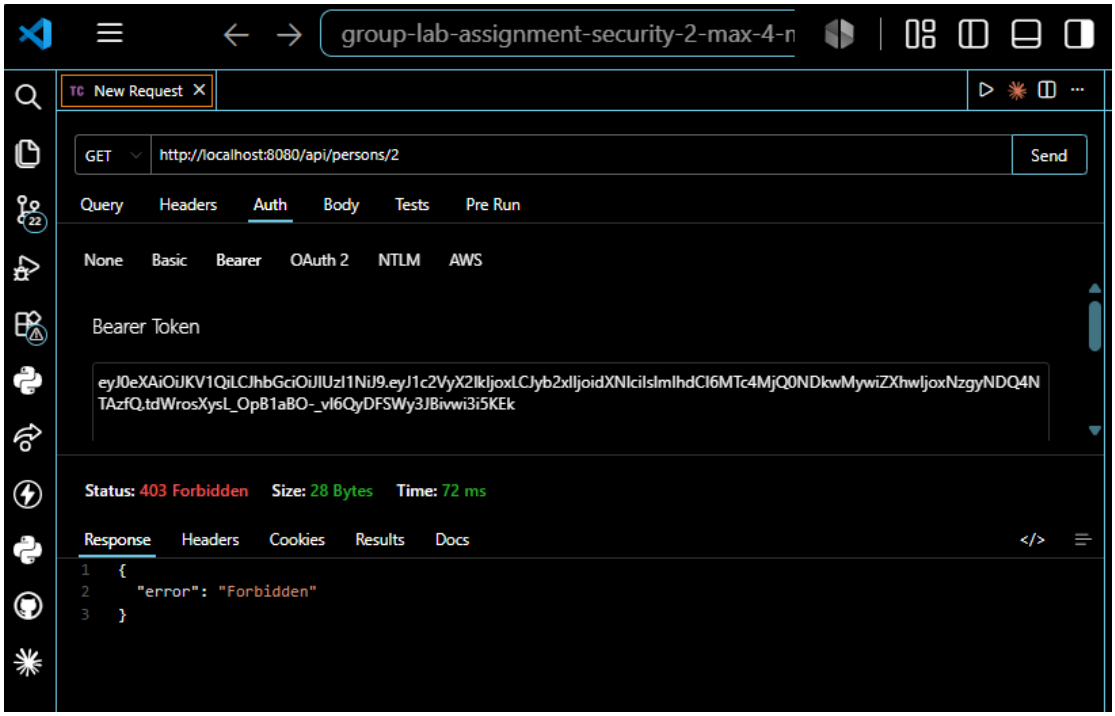


AFTER FIX:

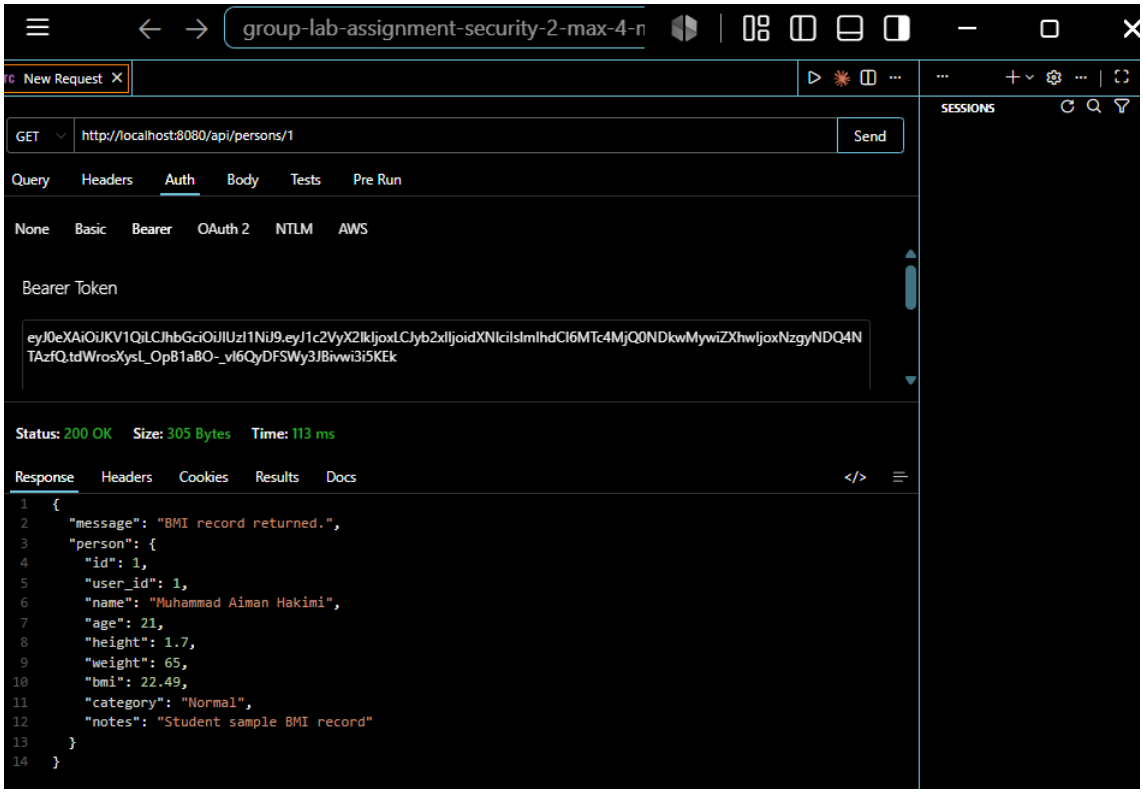
*We use Aiman accounts auth (Person 1)



When we try to get person 2 will get 403 response

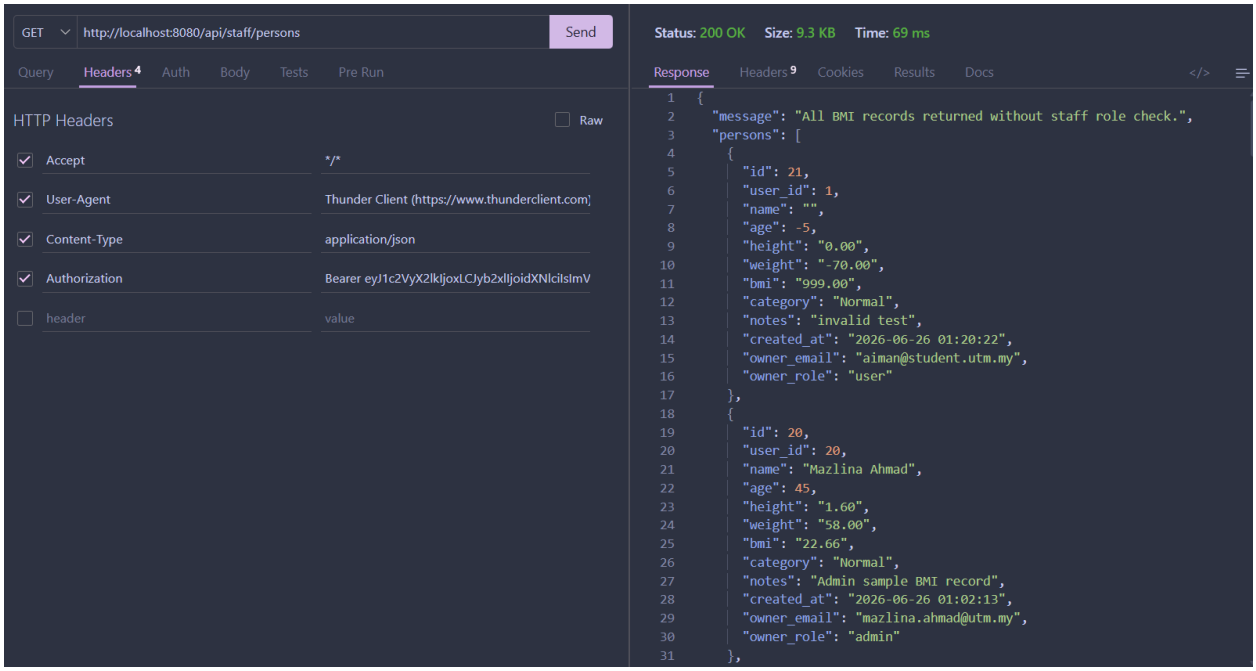


Aiman is person 1 so when get person 1 will return 200 OK response



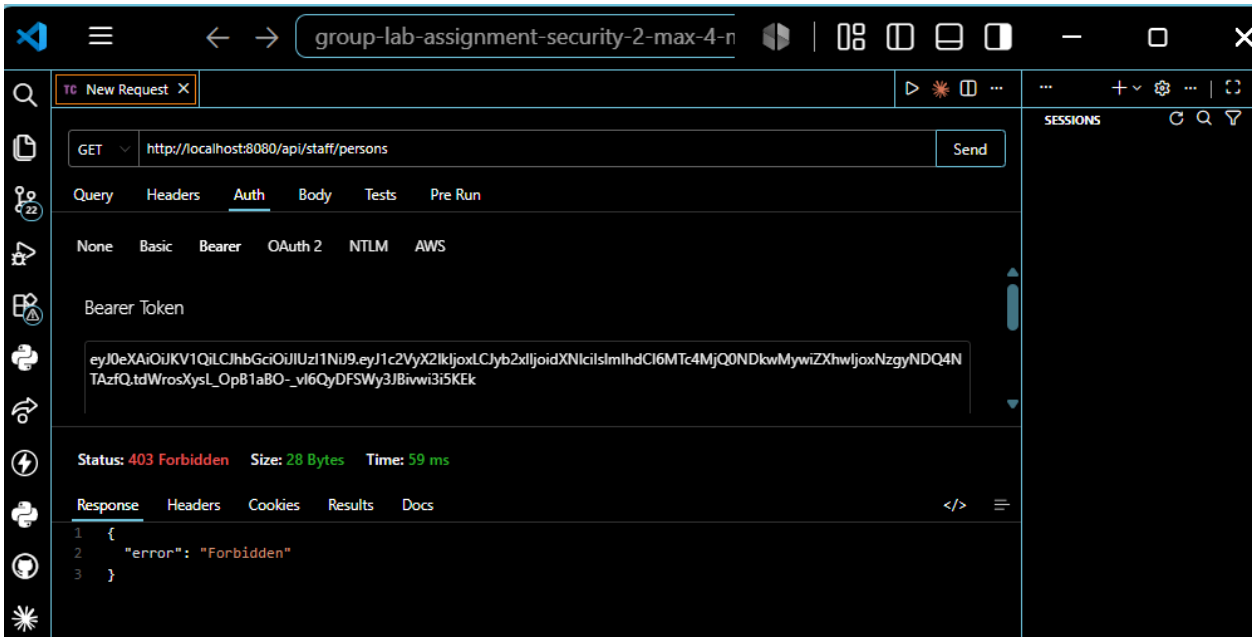
Test 3: Access staff route as normal user

BEFORE:

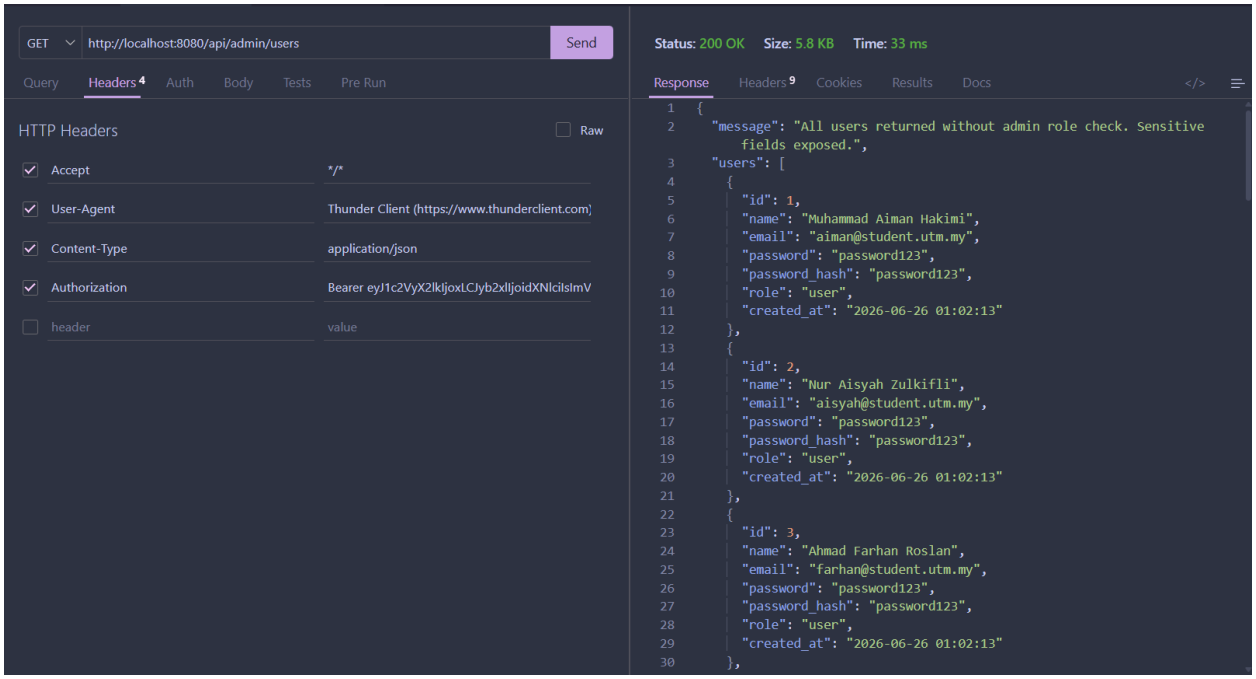


AFTER FIX:

*We use Aiman token as well (Aiman != staff)

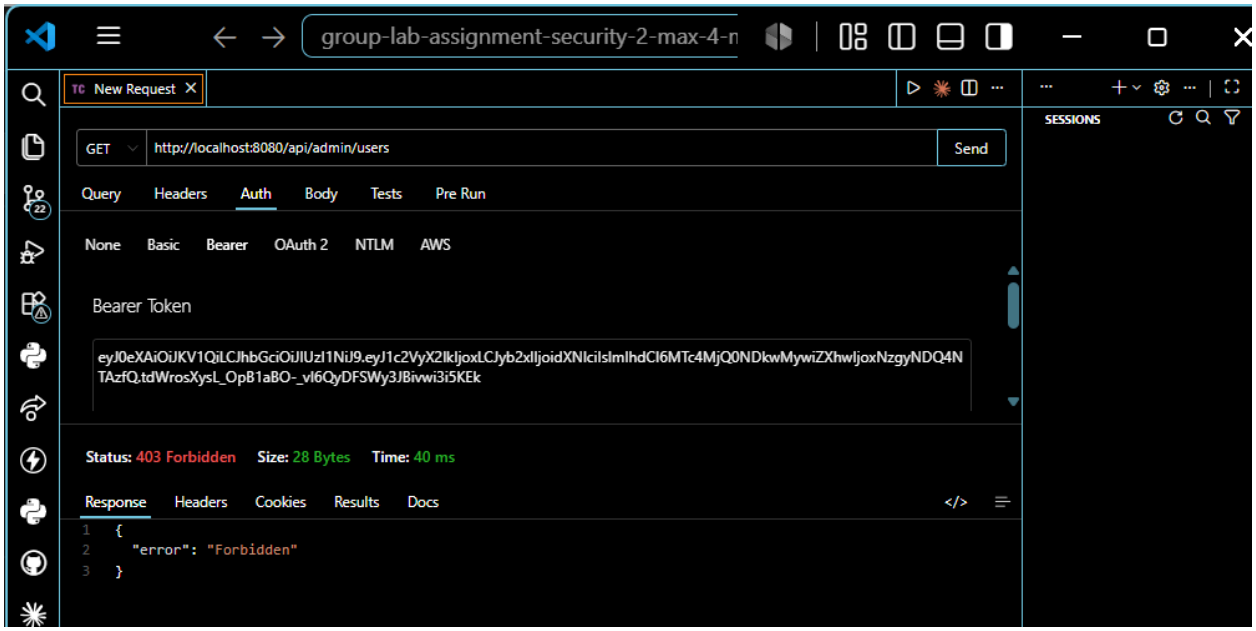


Test 4: Access admin route as normal user



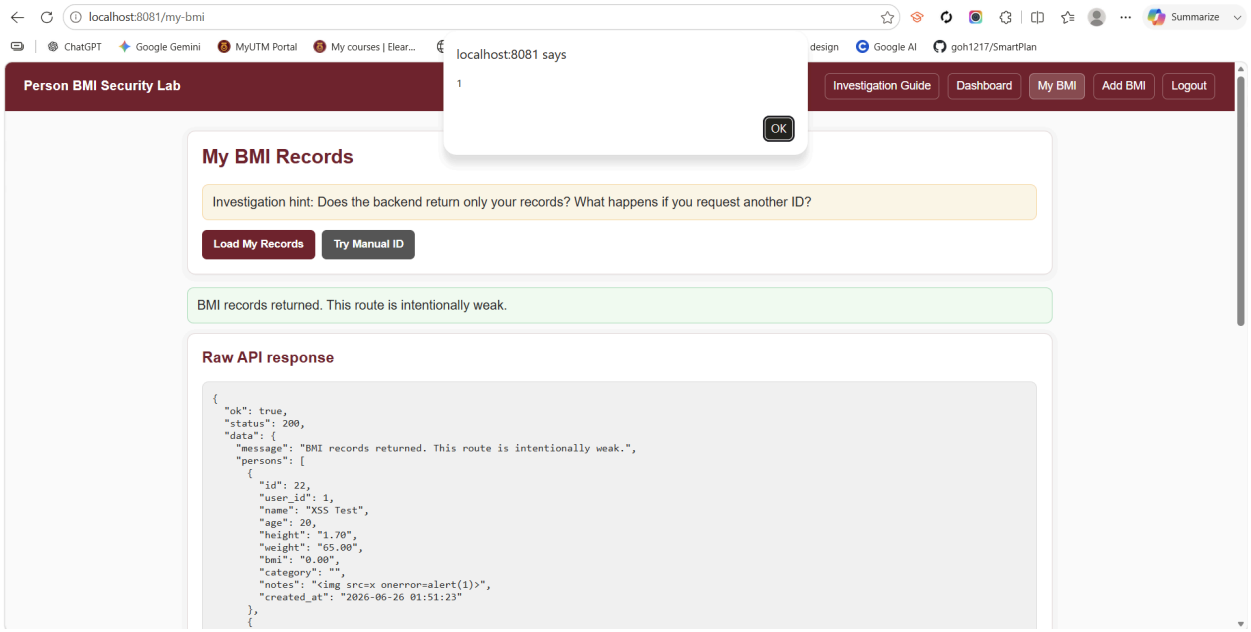
AFTER FIX:

*We use Aiman token as well (Aiman != admin)

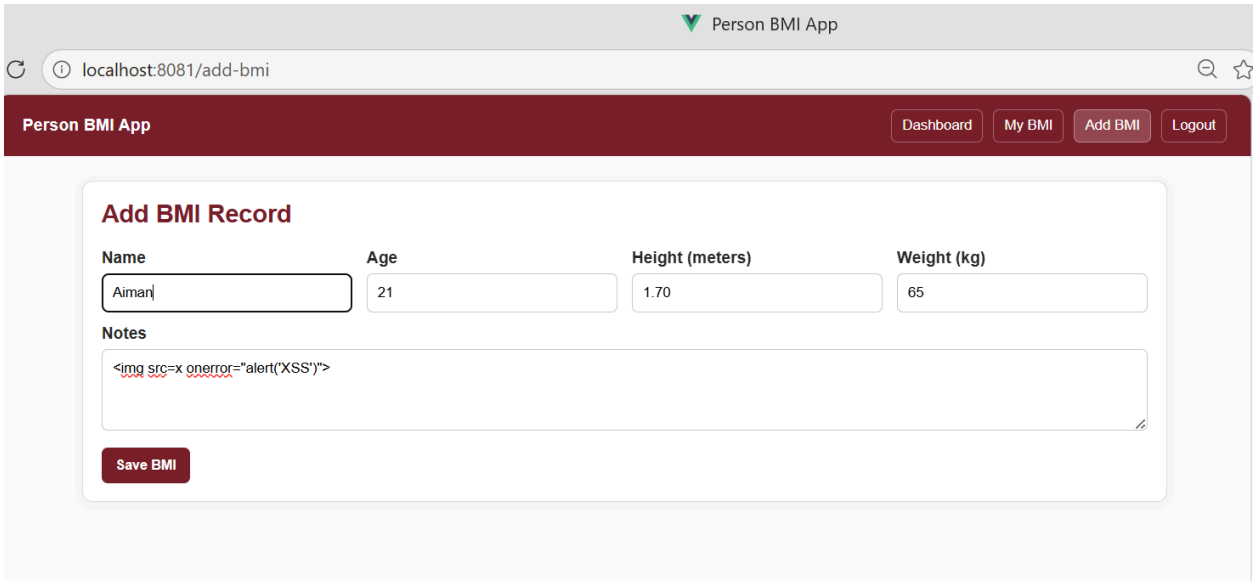


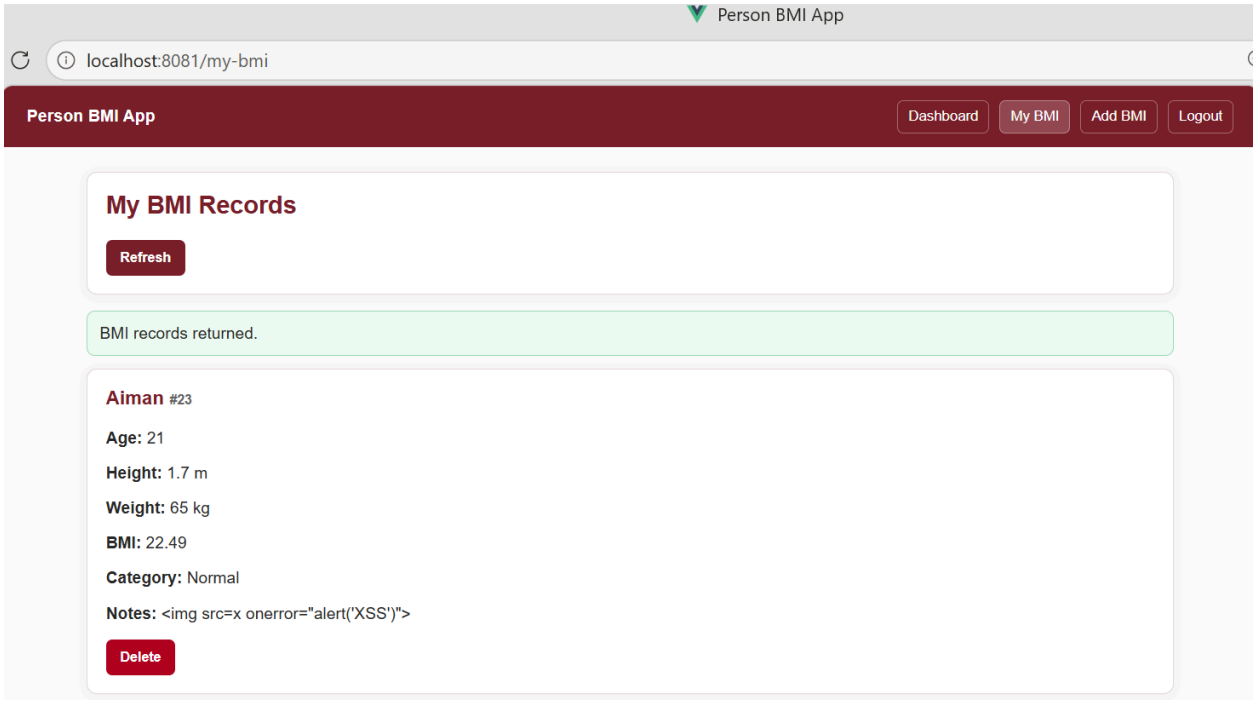
Test 5: Submit XSS payload in notes

BEFORE:



AFTER FIX:

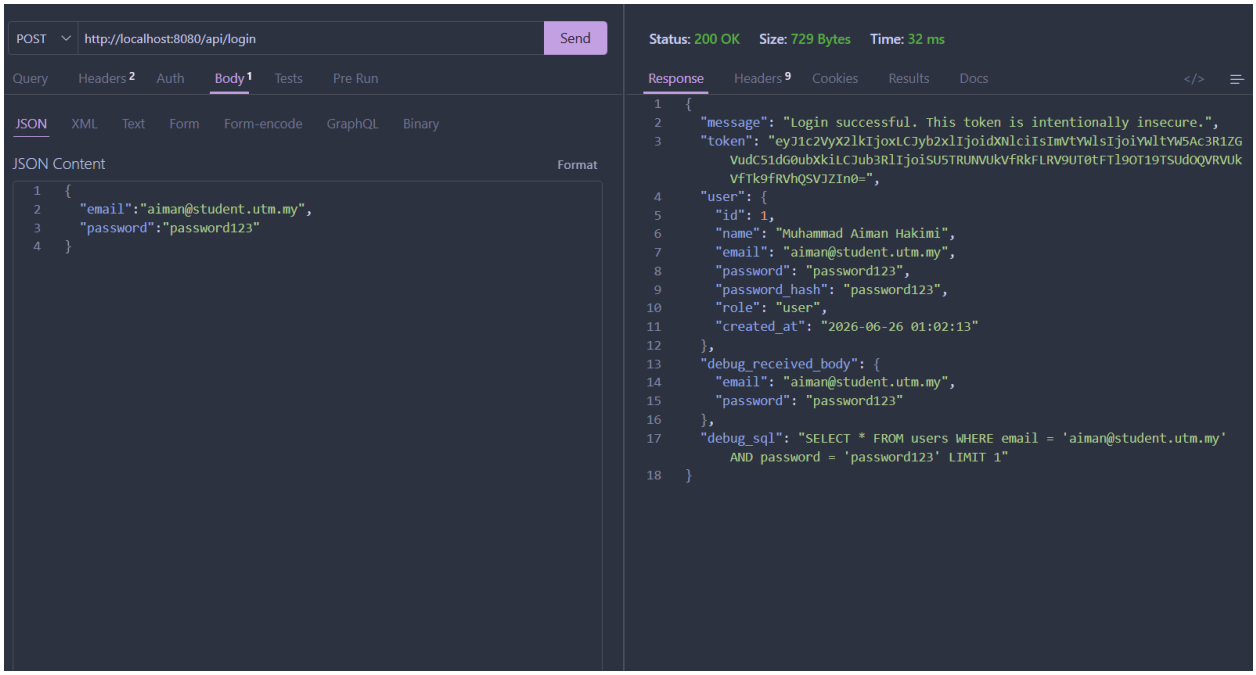




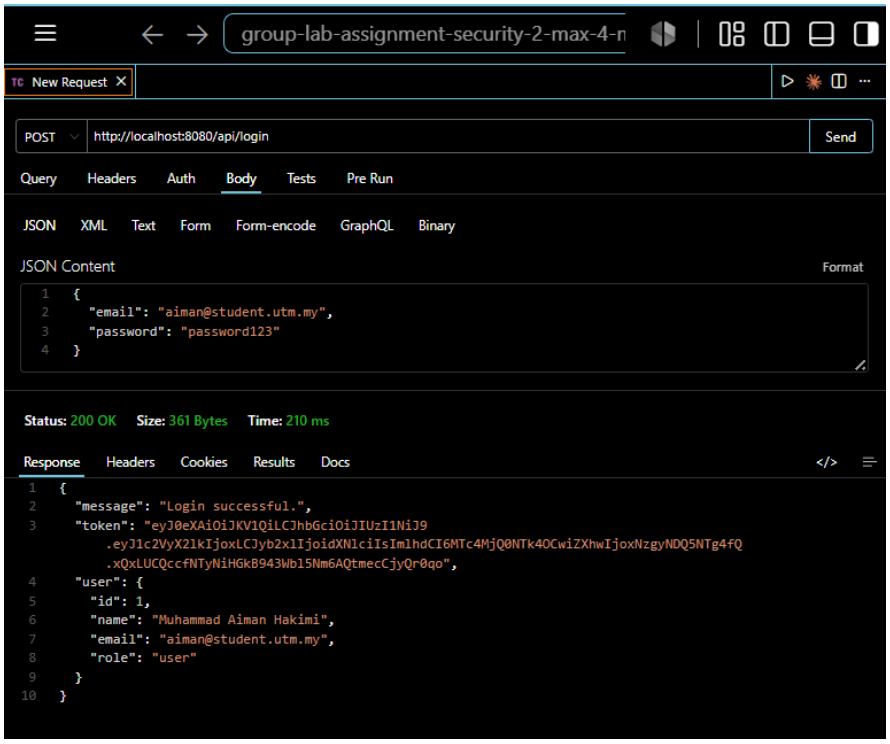
Before fix, v-html would have tried to run the script and you might see an alert. After fix, Vue {{ person.notes }} escapes HTML, so it only shows text.

Test 6: Check API response

BEFORE:



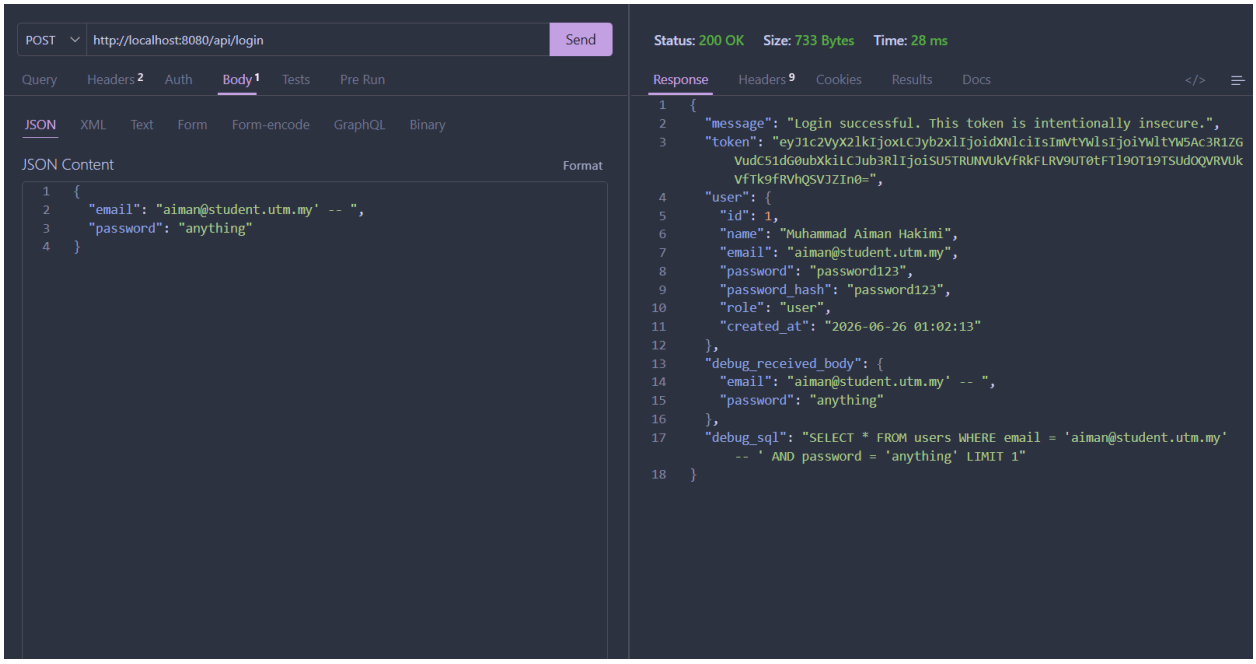
AFTER:



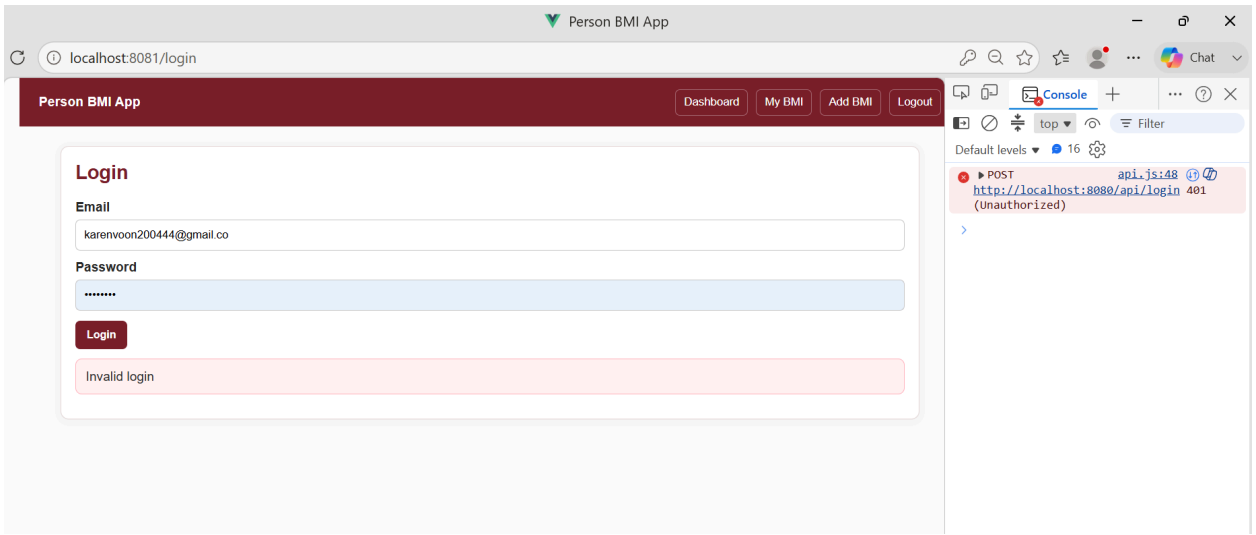
After fixing no password, password_hash, or debug_sql shown in response.

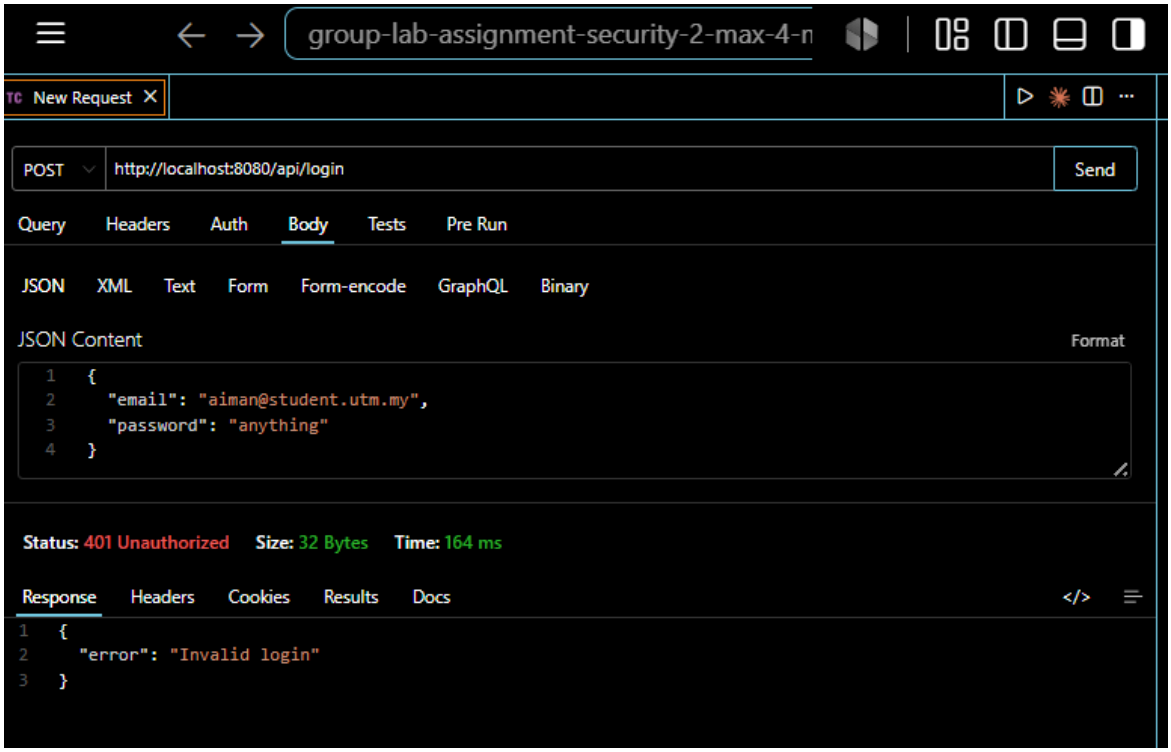
Test 7: Try SQL Injection input

BEFORE:



AFTER FIX:

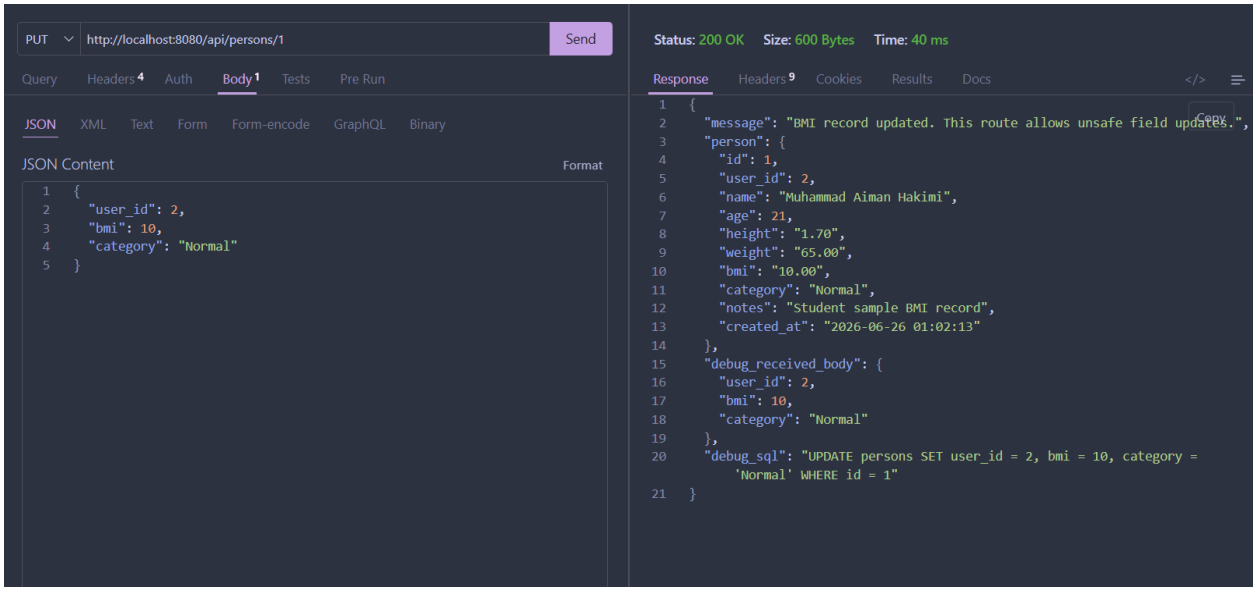




After fix will show 401 Invalid login when user key in wrong email or password.

Test 8: Try to update protected fields

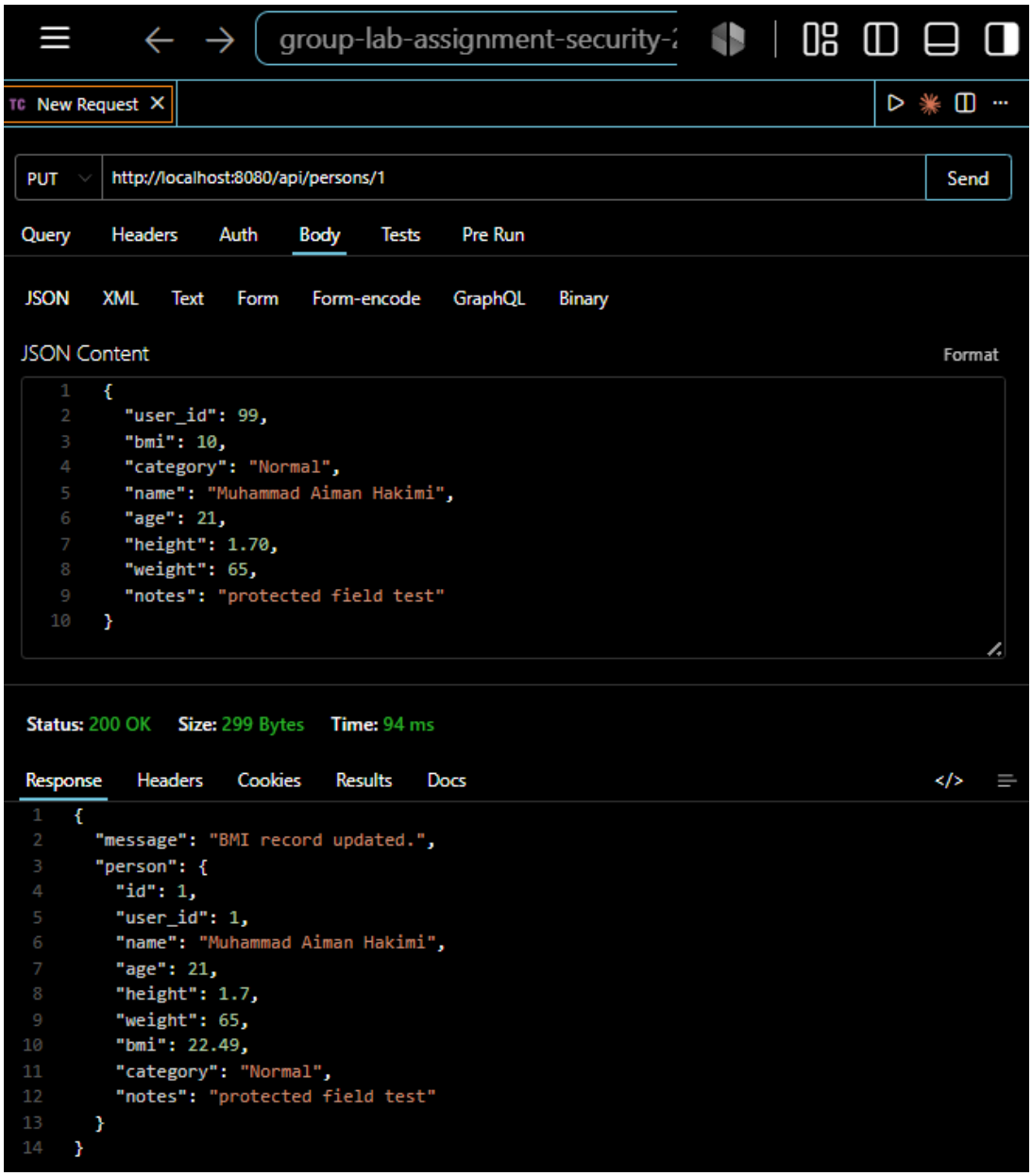
BEFORE:



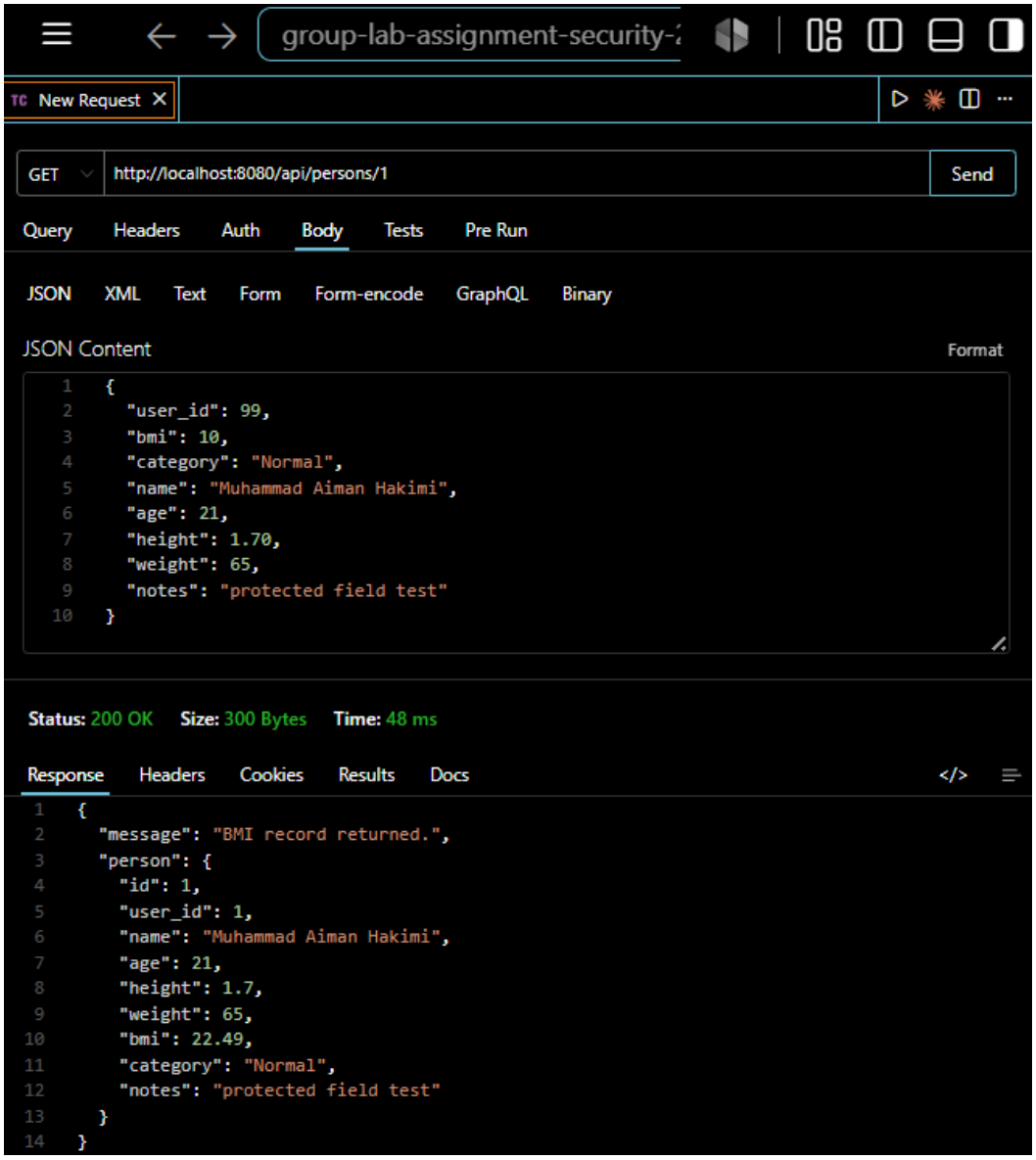
AFTER FIX:

*Use Aiman token

→PUT with protected fields



→GET to verify ignored



Response showing "user_id" is 1 not 99, bmi is 22.49 not 10.

5. API Endpoints Tested

Method	Endpoint	Purpose
GET	`/api/health`	API health check
POST	`/api/login`	Login authentication
POST	`/api/register`	User registration
GET	`/api/profile`	Retrieve current user profile
GET	`/api/persons`	Retrieve BMI records
POST	`/api/persons`	Create BMI record
GET	`/api/persons/{id}`	Retrieve BMI record by ID
PUT	`/api/persons/{id}`	Update BMI record
DELETE	`/api/persons/{id}`	Delete BMI record
GET	`/api/staff/persons`	Staff-only BMI records
GET	`/api/admin/users`	Admin-only user list
PUT	`/api/admin/users/{id}/role`	Update user role
DELETE	`/api/admin/persons/{id}`	Admin delete BMI record

6. GitHub commit summary

Commit ID	Commit Message	Description
66c3d46	Set up insecure frontend and backend projects	Initialized the intentionally vulnerable Person BMI frontend and backend applications used as the baseline for the security investigation.
0184919	Validate BMI input, hash passwords, and use PDO prepared statements	Added server-side validation for BMI input, implemented secure password hashing using `password_hash()` and `password_verify()`, and replaced vulnerable SQL string concatenation with PDO prepared statements to prevent SQL Injection.
cf2cc9c	Add JWT authentication and protected routes	Implemented JWT-based authentication, added authentication middleware, protected API endpoints, and replaced the insecure fake token mechanism.
8af92fb	Add RBAC ownership and protected field controls	Implemented Role-Based Access Control (RBAC), owner-based authorization (IDOR protection), and prevented users from modifying protected fields such as `user_id`, `bmi`, and `category`.

e7b904c	Sanitize API responses, prevent XSS, and secure error handling	Removed sensitive information (e.g., `password` and `password_hash`) from API responses, replaced `v-html` with safe output rendering to prevent XSS, and implemented generic error handling to avoid exposing SQL errors or server file paths.
14f68f3	Update README.md	Updated the project documentation with setup instructions, implemented security improvements, testing procedures, and usage information.

7. Reflection answers

1. Which weakness is the most dangerous? Why?
   SQL Injection is the most dangerous weakness because it can bypass authentication, manipulate database queries, retrieve sensitive information, and potentially compromise the entire application database.

2. Which weakness is caused by trusting the frontend?
   Mass Assignment is caused by trusting the frontend because the backend accepted protected fields such as user_id, BMI, and category directly from client requests without verification.

3. Which weakness is caused by missing backend validation?
   The acceptance of invalid BMI data, including negative weight, zero height, and empty names, was caused by missing backend validation.

4. Which weakness is caused by missing authorization?
   Broken Object Level Authorization (IDOR) and Broken Function Level Authorization (RBAC) were caused by missing authorization checks. Users could access other users' records and administrator routes without proper permission.

5. Which weakness can expose private data?
   Sensitive Data Exposure and IDOR can expose private data. Sensitive API responses may reveal password hashes, while IDOR allows unauthorized users to access other users' personal BMI records.

6. Which weakness can affect the browser?
   Cross-Site Scripting (XSS) affects the browser because malicious JavaScript can execute in a user's browser, allowing attackers to steal session information, manipulate webpage content, or perform unauthorized actions on behalf of the user.

7. Which weakness was easiest to find? Why?
   The easiest weakness to find was the missing backend validation because the application accepted invalid inputs such as empty names, negative ages, zero height, and negative weight. These problems were immediately visible during normal testing, making the vulnerability easy to identify.

8. Which weakness was most dangerous? Why?
   The most dangerous weakness was SQL Injection because it allowed attackers to manipulate SQL queries, bypass authentication, access confidential information, and potentially modify or delete database records. A successful SQL Injection attack could compromise the entire system.

9. Why is frontend validation not enough?
   Frontend validation only improves the user experience and can easily be bypassed. Attackers can send requests directly to the backend using API testing tools, so the backend must always validate every request before processing it.

10. Why should BMI calculation be performed at the backend?
    BMI should be calculated on the backend because it is derived from height and weight. Allowing users to submit their own BMI value could lead to incorrect or manipulated data. Calculating BMI on the server ensures consistency and data integrity.

11. What is the difference between authentication and authorization?
    Authentication verifies the identity of a user, while authorization determines what resources or actions that authenticated user is allowed to access. Authentication happens first, followed by authorization.

12. What is the difference between RBAC and owner-based access control?
    RBAC grants permissions based on user roles, such as User, Staff, or Admin. Owner-based access control ensures users can only access or modify resources that belong to them, even if they have the same role as other users.

13. Why is JWT alone not enough to protect all data?
    JWT only confirms the user's identity after login. It does not verify whether the user owns a resource or has permission to perform specific actions. The backend must still perform authorization checks such as RBAC and ownership verification.

14. Why should API responses exclude password_hash?
    Password hashes are sensitive information and should never be exposed to clients. Although hashed passwords cannot be directly read, exposing them increases security risks and provides attackers with unnecessary information.

15. Why is Vue route guard not enough to protect backend routes?
    Vue route guards only restrict access within the frontend application. Attackers can bypass the frontend completely by sending requests directly to backend APIs. Therefore, backend authentication and authorization must always be implemented.

16. How did you prove that your security fix works?

    The fixes were verified by repeating all security tests using Thunder Client and the frontend application. Previously successful attacks were rejected with appropriate HTTP status codes such as 400 Bad Request, 401 Unauthorized, and 403 Forbidden.

17. What security rule is missing here?
    The missing security rule is proper server-side validation and authorization. Every request should be validated, authenticated, and authorized before any database operation is performed.

18. Is this a frontend problem or backend problem?
    This is primarily a backend problem because backend security cannot rely on the frontend. All critical security checks must be enforced on the server.

19. Can this be bypassed using Postman?
    Yes. If security exists only in the frontend, attackers can bypass it by sending direct HTTP requests using Postman, Thunder Client, or similar API tools.

20. What does the JWT prove?
    A JWT proves that the user has successfully authenticated and contains trusted identity information such as the user ID and role. However, it does not automatically authorize access to every resource.

21. Does login alone mean the user can access everything?
    No. Login only authenticates the user. The backend must still verify permissions before allowing access to protected resources or performing sensitive operations.

22. Who owns this BMI record?
    The owner of a BMI record is the authenticated user whose **user_id** is associated with that record. Only the owner or an authorized administrator should be allowed to view, update, or delete it.

23. Which roles should access this route?
    Normal users should only access their own BMI records. Staff should access staff-specific routes, while administrator routes should only be accessible by users with the Admin role.

24. What fields should the frontend be allowed to update?
    The frontend should only update editable fields such as name, age, height, weight, and notes. Sensitive fields such as user_id, role, BMI, and category should not be accepted from the client.

25. What fields should be calculated by backend?
    The backend should calculate or assign the BMI value, BMI category, authenticated user ID, and system-generated timestamps. These values should never be supplied by the client.

26. How do you prove the fix works?

    The fixes were verified by repeating all previously successful attacks. Invalid requests were rejected, unauthorized access returned 401 or 403 errors, SQL Injection no longer bypassed login, XSS payloads were displayed as plain text, and protected fields could no longer be modified.